

P3107R0

Permit an efficient implementation of `std::print`

Published Proposal, 2024-01-28

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Introduction

2 Problems

3 Proposal

4 Performance

5 Implementation

6 Wording

References

Informative References

§ 1. Introduction

C++23 introduced a new formatted output facility, `std::print` ([P2093]). It was defined in terms of formatting into a temporary `std::string` to simplify the specification and to clearly indicate the requirement for non-interleaved output. Unfortunately, it was discovered that this approach does not allow for a more efficient implementation strategy, such as writing directly to a stream buffer under a lock, as reported in [LWG4042]. This paper proposes a solution to address this shortcoming.

§ 2. Problems

As reported in [LWG4042], `std::print/std::vprint*` is currently defined in terms of formatting into a temporary `std::string`, e.g. `[print.fun]`:

```
void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
```

Preconditions: `stream` is a valid pointer to an output C stream.

Effects: Writes the result of `vformat(fmt, args)` to `stream`.

Throws: Any exception thrown by the call to `vformat` ([[format.err.report](#)]). `system_error` if writing to `stream` fails. May throw `bad_alloc`.

This prohibits a more efficient implementation strategy of formatting directly into a stream buffer under a lock (`flockfile/funlockfile` in POSIX, [[STDIO-LOCK](#)]) like C `stdio` and other formatting facilities do.

The inability to achieve this with the current wording stems from the observable effects: throwing an exception from a user-defined formatter currently prevents any output from a formatting function, whereas with the direct method, the output written to the stream before the exception occurred is preserved. Most errors are caught at compile time, making this situation uncommon. The current behavior can be easily replicated by explicitly formatting into an intermediate string or buffer.

Another problem is that such double buffering may require unbounded memory allocations, making `std::print` unsuitable for resource-constrained applications creating incentives for continued use of unsafe APIs. In the direct method, there are usually no memory allocations.

§ 3. Proposal

The current paper proposes expressing the desire to have non-interleaved output in a way that permits a more efficient implementation similar to `printf`'s. It is based on the locking mechanism provided by C streams, quoting Section 7.21.2 Streams of the C standard ([\[N2310-STREAMS\]](#)):

7 Each stream has an associated lock that is used to prevent data races when multiple threads of execution access a stream, and to restrict the interleaving of stream operations performed by multiple threads. Only one thread may hold this lock at a time. The lock is reentrant: a single thread may hold the lock multiple times at a given time.

8 All functions that read, write, position, or query the position of a stream lock the stream before accessing it. They release the lock associated with the stream when the access is complete.

As shown in [Performance](#), this can give more than 20% speed up even compared to writing to a stack-allocated buffer.

All of the following languages use an implementation consistent with the current proposal (no intermediate buffering):

- C (`printf`)
- Rust (`println!`)
- Java (`System.out.format`)

IOStreams don't provide atomicity which is even weaker than the guarantees provided by these languages and the current proposal.

§ 4. Performance

The following benchmark demonstrates the difference in performance between different implementation strategies using the reference implementation of `print` from [\[FMT\]](#). This benchmark is based on the one from [\[P2093\]](#) but modified to avoid the small string optimization effects. It formats a simple message and prints it to the output stream redirected to `/dev/null`. It uses the Google Benchmark library [\[GOOGLE-BENCH\]](#) to measure timings:

```
#include <cstdio>
#include <benchmark/benchmark.h>
#include <fmt/format.h>
```

```

void printf(benchmark::State& s) {
    while (s.KeepRunning())
        std::printf("The answer to life, the universe, and everything is %d.\n", 42);
}
BENCHMARK(printf);

void vprint_string(fmt::string_view fmt, fmt::format_args args) {
    auto s = fmt::vformat(fmt, args);
    int result = fwrite(s.data(), 1, s.size(), stdout);
    if (result < s.size()) throw fmt::format_error("fwrite error");
}

template <typename... T>
void print_string(fmt::format_string<T...> fmt, T&&... args) {
    vprint_string(fmt, fmt::make_format_args(args...));
}

void print_string(benchmark::State& s) {
    while (s.KeepRunning()) {
        print_string("The answer to life, the universe, and everything is {}.\n", 42);
    }
}
BENCHMARK(print_string);

void vprint_stack(fmt::string_view fmt, fmt::format_args args) {
    auto buf = fmt::memory_buffer();
    fmt::vformat_to(std::back_inserter(buf), fmt, args);
    int result = fwrite(buf.data(), 1, buf.size(), stdout);
    if (result < buf.size()) throw fmt::format_error("fwrite error");
}

template <typename... T>
void print_stack(fmt::format_string<T...> fmt, T&&... args) {
    vprint_stack(fmt, fmt::make_format_args(args...));
}

void print_stack(benchmark::State& s) {
    while (s.KeepRunning()) {
        print_stack("The answer to life, the universe, and everything is {}.\n", 42);
    }
}
BENCHMARK(print_stack);

```

```

void print_direct(benchmark::State& s) {
    while (s.KeepRunning())
        fmt::print("The answer to life, the universe, and everything is {}.\\n",
    }
    BENCHMARK(print_direct);

    BENCHMARK_MAIN();

```

Here `print_string` formats into a temporary string, `print_stack` formats into a buffer allocated on stack and `print_direct` formats directly into the C stream buffer under a lock. `printf` is included for comparison.

The benchmark was compiled with Apple clang version 15.0.0 (clang-1500.1.0.2.5) with `-O3 -DNDEBUG` and run on macOS 14.2.1 with M1 Pro CPU. Below are the results:

```

Run on (8 X 24 MHz CPU s)
CPU Caches:
  L1 Data 64 KiB
  L1 Instruction 128 KiB
  L2 Unified 4096 KiB (x8)
Load Average: 5.03, 3.99, 3.89
-----
Benchmark                Time                CPU    Iterations
-----
printf                   81.8 ns             81.5 ns      8496899
print_string             88.5 ns             88.2 ns      7993240
print_stack              63.8 ns             61.9 ns     11524151
print_direct             51.3 ns             51.0 ns     13846580

```

Note that estimated CPU frequency is incorrect.

On Linux(Ubuntu 22.04.3 LTS) with gcc 11.4.0, glibc/libstdc++ and Intel Core i9-9900K CPU the results are similar except that `printf` is slightly faster than `print` with the stack-allocated buffer optimization:

```

Run on (16 X 3600 MHz CPU s)
CPU Caches:
  L1 Data 32 KiB (x8)
  L1 Instruction 32 KiB (x8)
  L2 Unified 256 KiB (x8)

```

```
L3 Unified 16384 KiB (x1)
Load Average: 0.00, 0.00, 0.00
```

Benchmark	Time	CPU	Iterations
printf	52.1 ns	52.1 ns	13386398
print_string	65.7 ns	65.7 ns	10674838
print_stack	55.8 ns	55.8 ns	12535414
print_direct	46.3 ns	46.3 ns	15087266

Direct output is 42-72% faster than writing to a temporary string and 21-24% faster than writing to a stack-allocated buffer on this benchmark.

§ 5. Implementation

This proposal has been implemented in the open-source {fmt} library ([\[FMT\]](#)) bringing major performance improvements.

§ 6. Wording

Modify [\[print.fun\]](#) as indicated:

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

Preconditions: `stream` is a valid pointer to an output C stream.

Effects: ~~The function initializes an automatic variable via~~

```
string out = vformat(fmt, args);
```

Let `out` denote the the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt`.

Locks `stream`. If `stream` refers to a terminal capable of displaying Unicode, writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it. Otherwise writes `out` to `stream` unchanged. If the native Unicode API is used, the function flushes `stream` before writing `out`. Releases the lock.

...

```
void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
```

Preconditions: `stream` is a valid pointer to an output C stream.

Effects: ~~Writes the result of `vformat(fmt, args)` to `stream`.~~ Locks `stream`, writes the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt` to `stream` and releases the lock.

Throws: Any exception thrown by the call to `vformat` ([`format.err.report`]). `system_error` if writing to `stream` fails. May throw `bad_alloc`.

...

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. *The {fmt} library*. URL: <https://github.com/fmtlib/fmt>

[GOOGLE-BENCH]

Google Benchmark: A microbenchmark support library. URL: <https://github.com/google/benchmark>

[LWG4042]

LWG Issue 4042: `std::print` should permit an efficient implementation. URL: <https://cplusplus.github.io/LWG/issue4042>

[N2310-STREAMS]

7.21.2 Streams. ISO/IEC 9899:202x. Programming languages — C. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf#page=233>

[P2093]

Victor Zverovich. *Formatted output*. URL: <https://wg21.link/p2093>

[STDIO-LOCK]

The Open Group Base Specifications Issue 7, 2018 edition. IEEE Std 1003.1-2017. flockfile, frylockfile, funlockfile - stdio locking functions. URL: <https://pubs.opengroup.org/onlinepubs/9699919799/functions/flockfile.html>